Live Demo Test Guide

What the Doctor Might Ask You to Do In Real Time

Fake News Detection Project · CSE 589 Neural Networks · 12 Practical Scenarios

Just like your database project where the doctor asked you to change something and see the effect, he can do the same here. Each scenario below shows exactly what he might say, what you should do step-by-step, and what the expected result looks like.

Scenario 1 — *Backend startup*

Start the backend and show me it is running

"Run your backend and prove the model loaded."

WHAT YOU DO:

```
# Open a terminal in the backend folder:
cd D:\fake-news-detection\backend

# Start the server using the deep-learning virtual environment:
..\ .venv-dl\Scripts\uvicorn.exe main:app --host 0.0.0.0 --port 8000

# Then open this URL in any browser:
http://localhost:8000/health
```

EXPECTED RESULT:

Terminal shows: "nn_model.keras loaded successfully" and "Application startup complete". Browser shows: {"status": "healthy"}

Tip: Also open http://localhost:8000/docs — FastAPI auto-generates a full interactive API page. Show the doctor this page to impress him.

Scenario 2 — *API test*

Send a prediction — show me a fake article classified

"Type a fake headline and show me what your model says."

WHAT YOU DO (option A — browser /docs):

Go to http://localhost:8000/docs → click POST /predict → Try it out → paste text → Execute.

WHAT YOU DO (option B — PowerShell):

```
Invoke-WebRequest -Uri "http://localhost:8000/predict" `
-Method POST `
-ContentType "application/json" `
-Body '{"text": "Obama secretly built underground tunnels to hide aliens from the public, leaked documents show."}' | Select-Object -Expand Content
```

EXPECTED RESULT:

> {"prediction": "Fake News", "confidence": 0.85}

NOW TRY A REAL ARTICLE:

```
{"text": "Scientists confirm climate change is accelerating according to new peer-reviewed research."}
```

> {"prediction": "Factual News", "confidence": 0.XX}

Scenario 3 *Error handling*

Send empty text — show me error handling

"What happens if someone sends you an empty request?"

WHAT YOU DO:

```
Invoke-WebRequest -Uri "http://localhost:8000/predict" `
-Method POST -ContentType "application/json" `
-Body '{"text": ""}' | Select-Object -Expand Content
```

EXPECTED RESULT:

> HTTP 400 Bad Request → {"detail": "Input text cannot be empty"}

WHERE THIS IS IN THE CODE: backend/main.py line 83 — if not raw: raise HTTPException(status_code=400, detail="Input text cannot be empty")

Scenario 4 *Live code change*

Change the confidence threshold and show the effect

"Change when the model decides something is fake — make it stricter."

WHAT YOU DO:

Open backend/main.py and find line 93. Change 0.5 to 0.7:

```
# BEFORE (line 93):
label = "Fake News" if prob >= 0.5 else "Factual News"
```

```
# AFTER:
label = "Fake News" if prob >= 0.7 else "Factual News"
```

Restart the server, send the same article again.

EXPECTED RESULT:

> Articles with model probability between 0.50 and 0.69 now show 'Factual News' instead of 'Fake News'. The model is now stricter — it only flags something as fake when it is at least 70% confident.

> Remember to change it back to 0.5 after the demo. Explain: 0.5 is standard for balanced datasets.

Scenario 5 *Live code change*

Change the response labels — 'FAKE' instead of 'Fake News'

"Change the output format. I want to see FAKE and REAL instead."

WHAT YOU DO:

Open backend/main.py, find line 93, change both label strings:

```
# BEFORE:
label = "Fake News" if prob >= 0.5 else "Factual News"

# AFTER:
label = "FAKE" if prob >= 0.5 else "REAL"
```

Restart and send any prediction request.

EXPECTED RESULT:

> {"prediction": "FAKE", "confidence": 0.85} ← label changed

Scenario 6 *Add a new field*

Add the raw probability to the API response

"I want to see the exact number the model outputs, not just the label."

WHAT YOU DO — Step 1: open backend/schemas.py and add probability field:

```
# BEFORE:
class PredictResponse(BaseModel):
prediction: str
confidence: float

# AFTER:
class PredictResponse(BaseModel):
prediction: str
```

```
confidence: float
probability: float # ← add this line
```

Step 2: open backend/main.py and update the return statement:

```
# BEFORE:
return PredictResponse(prediction=label, confidence=confidence)

# AFTER:
return PredictResponse(prediction=label, confidence=confidence,
probability=round(prob, 4))
```

Restart the server and send a request.

EXPECTED RESULT:

{"prediction": "Fake News", "confidence": 0.85, "probability": 0.8523}

Tip: The probability is the raw Sigmoid output (0 to 1). Confidence is max(prob, 1-prob) so it always reads as a positive certainty score.

Scenario 7

Text preprocessing

Add a word to the stopwords list and show the cleaning effect

"The word trump appears too often. Remove it from all articles."

WHAT YOU DO:

Open backend/services.py and add the word to the STOPWORDS set:

```
# Find the STOPWORDS set and add to it:
STOPWORDS = set("""
a about above ... yourselves
trump clinton obama ← add any words here
""".split())
```

To prove it works without restarting, run this in a Python shell:

```
from services import clean_text

# BEFORE adding trump to stopwords:
print(clean_text("Trump signed a bill about healthcare"))
# Output: trump signed bill healthcare

# AFTER adding trump to stopwords:
print(clean_text("Trump signed a bill about healthcare"))
```

```
# Output: signed bill healthcare ← trump is gone
```

EXPECTED RESULT:

> The word 'trump' no longer appears in any cleaned text sent to the model.

Scenario 8 *Model internals*

Show me what the model input actually looks like

"What exactly does the model receive? Show me the numbers."

WHAT YOU DO — run this in Python:

```
import joblib, numpy as np
from services import clean_text

vectorizer = joblib.load('vectorizer.pkl')

text = 'Scientists confirm climate change is real'
cleaned = clean_text(text)
print('Cleaned text:', cleaned)

X = vectorizer.transform([cleaned]).toarray()
print('Vector shape:', X.shape) # (1, 3401)
print('Non-zero features:', (X > 0).sum()) # how many words matched vocab
print('Sample values:', X[0, :10]) # first 10 TF-IDF scores
```

EXPECTED RESULT:

> Cleaned text: scientists confirm climate change real Vector shape: (1, 3401) Non-zero features: 4 Sample values: [0. 0. 0. 0. 0.023 0. 0. 0. 0. 0.]

Scenario 9 *Frontend-backend link*

What happens if the backend is stopped but the frontend is open?

"Stop your server. What does the user see on the website?"

WHAT YOU DO:

Stop the backend server (Ctrl+C in the terminal). Open the frontend (http://localhost:5173), type any article, click Analyze.

EXPECTED RESULT:

> The frontend shows a network error message — the POST /predict request fails because nothing is listening on port 8000. This shows the frontend and backend are separate services connected via HTTP.

WHERE THIS IS IN THE CODE: frontend/src/services/api.js — axios.post() to the backend URL. If the backend is down, axios throws a network error which is caught and displayed to the user.

Scenario 10

Model architecture

Show the model summary — how many parameters does it have?

"Tell me how many weights your neural network has and why."

WHAT YOU DO — run in Python:

```
import tensorflow as tf

model = tf.keras.models.load_model('nn_model.keras')

model.summary()
```

EXPECTED RESULT and EXPLANATION:

Layer 1: Dense(256) — 3401 inputs x 256 neurons + 256 biases = 871,168 params Layer 2: Dense(128) — 256 x 128 + 128 = 32,896 params Layer 3: Dense(64) — 128 x 64 + 64 = 8,256 params Layer 4: Dense(1) — 64 x 1 + 1 = 65 params TOTAL: 912,129 trainable parameters

Tip: If he asks why so many parameters: the first layer is large because the TF-IDF input has 3401 features. Each of the 256 neurons needs a weight for each feature. 3401 x 256 = 870,912 weights + 256 biases = 871,168.

Scenario 11

Extend the API

Add a new API endpoint — /model-info

"Add an endpoint that returns the model architecture info."

WHAT YOU DO — add this to backend/main.py:

```
@app.get("/model-info", summary="Model architecture info")

def model_info():

if model is None:

raise HTTPException(status_code=503, detail="Model not loaded")

return {

"name": model.name,

"total_params": model.count_params(),

"input_shape": str(model.input_shape),

"output_shape": str(model.output_shape),

"num_layers": len(model.layers)

}
```

Restart the server, visit http://localhost:8000/model-info

EXPECTED RESULT:

{"name": "FakeNewsDetector", "total_params": 912129, "input_shape": "(None, 3401)", "output_shape": "(None, 1)", "num_layers": 8}

Scenario 12 *Notebook demo*

Run the notebook and show me the training live

"Open the notebook and retrain the model in front of me."

WHAT YOU DO:

```
# 1. Open Jupyter in the project folder:
cd D:\fake-news-detection
.venv-dl\Scripts\jupyter.exe notebook

# 2. Open 02_Deep_Learning_Model.ipynb
# 3. Select kernel: Python 3.10 (deep-learning)
# 4. Click Kernel → Restart & Run All
```

WHAT TO POINT OUT DURING TRAINING:

Epoch 1: accuracy starts low (~50%) — random weights, model knows nothing. Epoch 5-8: accuracy jumps to 90%+ — model starts learning TF-IDF patterns. Epoch 23: EarlyStopping triggers — 'Restoring model weights from the end of best epoch: 8' Final: Test Accuracy = 97.50%, confusion matrix shows only 1 false negative. Last cell (Section 11) saves nn_model.keras and vectorizer.pkl to the backend folder.

Tip: While it trains, explain: 'Each epoch passes all 134 training samples through the network once, computes the loss, and runs backpropagation to update the 912K weights.'